
♦ `.then(res => res.json())`

```
.then(res => res.json())
```

Earlier:

```
fetch(...)
```

sent request to Spring Boot.

Now Spring Boot sends back:

```
{
  "result": "SPAM"
}
```

or

```
{
  "result": "HAM"
}
```

What is `res`?

```
res
```

is just a variable name.

Think:

```
Response from Spring Boot
```

stored inside:

```
res
```

What is `res.json()`?



Response arrives as raw HTTP response.

JavaScript cannot directly use:

```
{
  "result": "SPAM"
}
```

So:

```
res.json()
```

means:

```
Convert response  
into a JavaScript object
```

Example

Before:

```
'{"result":"SPAM}"'
```

After:

```
{  
  result:"SPAM"  
}
```

Now JavaScript can access:

```
data.result
```

later.

```
♦ .then(data => {
```

```
  .then(data => {
```

After:

```
res.json()
```

JavaScript gets:

```
{  
  result:"SPAM"  
}
```

This object is stored into:

```
data
```

So:

```
data
```

contains:

```
{  
  result:"SPAM"
```

```
}

```

```
◆ let box = document.getElementById("spam-box");
```

```
let box = document.getElementById("spam-box");
```

What is:

```
document
```

?

Predefined browser object. 

Represents the whole webpage.

What is:

```
getElementById()
```

?

Predefined method. 

Used to find HTML element by id.

Example:

If page contains:

```
<div id="spam-box">
</div>
```

then:

```
document.getElementById("spam-box")
```

finds it.

Think:

```
Find HTML element
whose id is spam-box
```

Result stored into:

```
box
```

So:

`box`

points to the spam display box.

◆ `if (!box)`

`if (!box)`



means:

If box does NOT exist

Example

Suppose Gmail page currently has:

No spam-box element

Then:

`box`

becomes:

`null`

So:

`!box`

becomes:

`true`

and code enters the if block.

Why?

Because we need to create the box first.

◆ `document.createElement("div")`

`box = document.createElement("div");`

What is:

`createElement()`

?

Predefined method.

Creates a new HTML element.

Example:

```
document.createElement("div")
```

creates:

```
<div></div>
```

inside memory.

Not yet visible.

Stored into:

```
box
```

So now:

```
box
```

contains:

```
<div></div>
```

```
◆ box.id = "spam-box";
```

```
box.id = "spam-box";
```

Adds id.

Now element becomes:

```
<div id="spam-box"></div>
```

Why?

So we can find it later using:

```
document.getElementById("spam-box")
```

```
◆ box.style.position = "fixed";
```

```
box.style.position = "fixed";
```

Adds CSS styling.

Meaning:

**Keep box fixed
on screen**

Even when scrolling.

```
♦ box.style.top = "10px";
```

```
box.style.top = "10px";
```

Means:

10 pixels from top

```
♦ box.style.right = "10px";
```

```
box.style.right = "10px";
```

Means:

10 pixels from right side

Result:

Spam: SPAM |

appears near top-right corner.

```
♦ box.style.background = "black";
```

```
box.style.background = "black";
```

Background color:

Black

```
♦ box.style.color = "white";
```

```
box.style.color = "white";
```

Text color:

White

♦ `box.style.padding = "10px";`

`box.style.padding = "10px";`

Adds space inside box.

Without padding:

Spam: SPAM

touches border.

With padding:

Nice spacing

♦ `box.style.borderRadius = "8px";`

`box.style.borderRadius = "8px";`

Rounds corners.

Without:



With:



♦ `box.style.zIndex = "9999";`

`box.style.zIndex = "9999";`

Means:

Keep this element
above almost everything

Without zIndex:

Spam box might hide behind Gmail.

With:

```
9999
```

it stays visible.

```
♦ document.body.appendChild(box);
```



```
document.body.appendChild(box);
```

What is:

```
appendChild()
```

?

Predefined method.

Means:

```
Add element  
to webpage
```

Before:

```
box
```

exists only in memory.

After:

```
appendChild(box)
```

browser inserts it into Gmail page.

Now user can see it.

```
♦ box.innerText = "Spam: " + data.result;
```

```
box.innerText = "Spam: " + data.result;
```

Suppose:

```
data
```

contains:

```
{  
  result: "SPAM"
```



```
}

```

Then:

```
data.result

```

returns:

```
SPAM

```

So:

```
"Spam: " + data.result

```

becomes:

```
"Spam: SPAM"

```

Then:

```
box.innerText

```

places text inside box.

Result visible on Gmail:

```
Spam: SPAM

```

or

```
Spam: HAM

```

```
♦ .catch(err => console.log("Error:", err));

```

```
.catch(err => console.log("Error:", err));

```



What if:

```
Spring Boot down

```

or

```
Flask not running

```

or

```
Network error

```

?

Then fetch fails.

Error gets stored in:

```
err
```

and printed:

```
Error: <details>
```

in browser console.

◆ COMPLETE FLOW

```
Gmail Email
↓
getEmailText()
↓
text
↓
fetch()
↓
Spring Boot
↓
Flask
↓
ML Model
↓
Prediction
{
  result:"SPAM"
}
↓
res.json()
↓
data
↓
Find spam-box
↓
Create it if missing
↓
Style it
↓
Add to webpage
↓
box.innerText
↓
Spam: SPAM
```

This is the entire journey from Gmail email text to the black popup box appearing on the screen. 